

the Monte Carlo Method

1 the Monte Carlo Method

- estimating integrals
- histograms of random number generators

2 Mean Time Between Failures

- life span of a multicomponent product
- running simulations

3 Servicing Requests

- estimating wait times
- running simulations

MCS 472 Lecture 3
Industrial Math & Computation
Jan Verschelde, 16 January 2026

the Monte Carlo Method

1 the Monte Carlo Method

- estimating integrals
- histograms of random number generators

2 Mean Time Between Failures

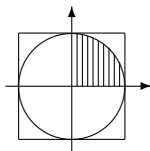
- life span of a multicomponent product
- running simulations

3 Servicing Requests

- estimating wait times
- running simulations

estimating π

- The area of the unit disk is π .



$$\frac{\pi}{4} = \int_0^1 \sqrt{1 - x^2} dx$$

- Generate random uniformly distributed points with coordinates

$$(x, y) \in [-1, +1] \times [-1, +1].$$

We count the generated (x, y) as a success when $x^2 + y^2 \leq 1$.

- Then $\pi \approx \frac{\text{total number of successes}}{\text{total number of trials}}$.

a Julia experiment

`rand(10)` returns a vector of 10 random numbers in $[0, 1]$.

Let us generate 10 million points (x, y) :

```
julia> N = 10^7;
```

```
julia> x = rand(N); y = rand(N);
```

```
julia> z = x.^2 + y.^2;
```

```
julia> 4*sum([Int32(t <= 1.0) for t in z])/N  
3.1418236
```

- The semicolon (;) suppresses the output.
- The dot before the exponent is for a componentwise operation.
- The True/False is converted to 1/0 by `Int32(t <= 1.0)`.

questioning the performance

The simulation on the previous slide is nice and short, but requires to keep three vectors of size N in main memory.

Exercise 1:

For the estimation of π , take N equal to 10^8 , 10^9 , etc.

- What are the resources in memory and time for increasing N ?
- How many decimal places in π do we gain as N increases?
- Rewrite the simulation so that not all points are in memory. Redo the experiments for increasing N . Report the improvements.

the newsboy problem

A newspaper seller has on average 100 customers a day as 5000 people are passing by the shop each day.

The seller has to buy each paper at 50 cents a copy, sells it at 75 cents, with no returns.

For each sold paper, the seller makes a profit of 25 cents, the loss is 75 cents for each unsold paper.

To maximize profit, how many copies should the seller buy?

applying Monte Carlo simulation

The sentence

A newspaper seller has on average 100 customers a day as 5000 people are passing by the shop each day.

translates into the following simulation:

- 1 Flip a loaded coin (with probability $100/5000$) five thousand times, say that heads comes up with probability $1/50$.
- 2 Count each head as a one, as a customer who buys a paper.

How many copies should the seller buy?

Run the simulation starting with 90, then 91, 92, 93, ..., to 110 and for each run, compute the profit.

Take the average for 365 days, for 365 simulations.

a Julia function

```
function newsboy()
    (maxprofit, maxidx) = (0, 0)
    for nbpapers = 90:110
        sumday = 0
        for day=1:365
            profit = -nbpapers*0.50
            nbsold = 0
            for passerby=1:5000
                if nbsold < nbpapers
                    if rand() <= 1.0/50
                        profit = profit + 0.75
                        nbsold = nbsold + 1
                    end
                end
            end
            sumday = sumday + profit
        end
    end
end
```

the function continued ...

Averaging daily profits:

```
    avgday = sumday/365
    println("profit from ", nbpapers, " : \$", avgday)
    if maxprofit < avgday
        maxprofit = avgday
        maxidx = nbpapers
    end
end
print(" maximum profit : \$", maxprofit)
println(" at ", maxidx)
end
```

running at the command prompt

```
$ julia newsboy.jl
profit from 90 : $21.94
profit from 91 : $22.23
profit from 92 : $22.05
profit from 93 : $22.35
profit from 94 : $22.17
profit from 95 : $22.38
profit from 96 : $22.52
profit from 97 : $22.16
profit from 98 : $22.22
profit from 99 : $21.72
profit from 100 : $22.24
profit from 101 : $21.92
profit from 102 : $22.11
profit from 103 : $21.43
profit from 104 : $21.23
profit from 105 : $21.43
profit from 106 : $20.98
profit from 107 : $20.12
profit from 108 : $20.14
profit from 109 : $19.53
profit from 110 : $19.26
maximum profit : $22.52 at 96
```

the baker's problem

Exercise 2:

A grocery store has a bake shop.

A loaf of bread costs 90 cents to make.

- When fresh, sold on the same day, it sells for \$1.40.
- The next day, the loaf sells for 70 cents.
- After two days, the loaf has to be discarded.

Given that of the 300 customers in the store,
on average 20 customers buy a loaf of bread.

To maximize profit, how many loafs of bread should the shop bake?

Justify your answer

by adapting the simulation for the newsboy problem.

the Monte Carlo Method

1 the Monte Carlo Method

- estimating integrals
- histograms of random number generators

2 Mean Time Between Failures

- life span of a multicomponent product
- running simulations

3 Servicing Requests

- estimating wait times
- running simulations

random number generators

Flipping a loaded coin in Julia:

```
julia> coin() = Int32(rand() <= 1/50)
```

coin (generic function with 1 method)

```
julia> flips = [coin() for i=1:10000];
```

```
julia> sum(flips)
```

186

```
julia> ans/10000
```

0.0186

```
julia> 1/50
```

0.02

making a histogram for `randn()`

The `randn(10)` returns a vector 10 normally distributed numbers of mean zero and standard deviation one.

Exercise 3:

Use `randn()` to generate one million random numbers.

- 1 How many of those generated numbers lie in $[-1, +1]$?
Is this normal?
- 2 Approximating the cumulative distribution function, count how many of the generated numbers are less than $-3 + 0.1k$, for k ranging from 0 to 60, making 61 counts.

the Monte Carlo Method

1 the Monte Carlo Method

- estimating integrals
- histograms of random number generators

2 Mean Time Between Failures

- life span of a multicomponent product
- running simulations

3 Servicing Requests

- estimating wait times
- running simulations

a multicomponent product

Given is a composite product of n components:

- The life span of the i -th component is normally distributed with mean μ_i and standard deviation σ_i , for $i = 1, 2, \dots, n$.
- The composite product fails if any of its components fails.

What is the average life span of the composite product?

a simulation algorithm

Input: $T = (T_1, T_2, \dots, T_n)$,

T_i samples from a normal distribution

with mean μ_i and standard deviation σ_i ,

$N = \#T_i, i = 1, 2, \dots, n$, number of samples.

Output: expected life span of the n -component product.

```
sum := 0
```

```
for  $i$  from 1 to  $N$  do
```

```
  for  $j$  from 1 to  $n$  do
```

```
     $t_j := T_{i,j}$ 
```

```
  end for
```

```
  sum := sum + min( $t_1, t_2, \dots, t_n$ )
```

```
end for
```

```
report sum/ $N$ 
```

This algorithm estimates an n -dimensional integral.

the Monte Carlo Method

1

the Monte Carlo Method

- estimating integrals
- histograms of random number generators

2

Mean Time Between Failures

- life span of a multicomponent product
- running simulations

3

Servicing Requests

- estimating wait times
- running simulations

simulations with a Julia function

We start with the documentation string of the function.

```
"""
```

```
    mtbf(means::Vector{Float64},  
         deviations::Vector{Float64},  
         N::Int,  
         verbose::Bool=false)
```

returns the expected lifespan and the standard deviation of a multicomponent product, given the means and standard deviations of its components in the two vectors on input, using N trials in the simulation.

If verbose, then each sample is written.

```
"""
```

the definition of the function

```
function mtbf(means::Vector{Float64},
              deviations::Vector{Float64}, N::Int,
              verbose::Bool=false)
    (m1, m2) = (0, 0) # moments
    sample = [0.0 for j=1:length(means)] # work space
    for i=1:N # run N simulations
        for j=1:length(means)
            sample[j] = means[j] + deviations[j]*randn()
        end
        if verbose # for debugging
            println("sample : ", sample)
        end
        lifespan = minimum(sample)
        m1 = m1 + lifespan/N # update the average
        m2 = m2 + lifespan*lifespan/N # update second moment
    end
    return (m1, sqrt(m2 - m1*m1)) # mean and deviation
end
```

The `randn()` has a mean of zero and standard deviation of one.

running a simulation

Assume we have three components:

- The means are 11, 12, and 13.
- The standard deviations are 1, 2, and 3.

Let us take 10,000 samples in a simulation.

```
(mu, sigma) = mtbf([11.0, 12.0, 13.0], [1.0, 2.0, 3.0], 10000)
println("expected life span : ", mu)
println("standard deviation : ", sigma)
```

which prints

```
expected life span : 10.099215794814308
standard deviation : 1.371475210671779
```

It is important to realize that the accuracy
is only about one or at most two decimal places...

backup components

Normal bodies have two kidneys, two lungs, two eyes, ...

Exercise 4:

Apply simulation to estimate the life span of a multicomponent product with backups, to answer the following:

- The product has three components, all with mean 10 and standard deviation 1.
- The second component is a backup of the first:

For the product to fail:

- ▶ the first two components both fail, or
- ▶ the third component fails.

Estimate the life span and standard deviation with at least 10,000 samples in a simulation.

the Monte Carlo Method

1

the Monte Carlo Method

- estimating integrals
- histograms of random number generators

2

Mean Time Between Failures

- life span of a multicomponent product
- running simulations

3

Servicing Requests

- **estimating wait times**
- running simulations

modeling checkout lines

Problem: predict the checkout time in a store.

This time depends on three parameters:

- 1 arrival times of jobs,
- 2 service times to handle requests,
- 3 number of processors.

We use the following distributions:

- 1 Arrival times are modeled by the Poisson distribution.
- 2 The service times depend on the size of each job, which we assume to follow a uniform or normal distribution.

the Poisson distribution

Let $\lambda > 0$, be a positive real number.

λ = expected number of events in a time interval assuming events occur independently

The probability that exactly k events happen:

$$f(k; \lambda) = \frac{\lambda^k e^{-\lambda}}{k!}.$$

Mean and standard deviation:

- λ is average,
- the standard deviation is $\sqrt{\lambda}$.

generating Poisson numbers

Input: λ , average #events per time interval.

Output: N , #events per time interval.

```
 $L := e^{-\lambda}; k := 0; p := 1;$   
do  
   $k := k + 1;$   
  draw  $u \in [0, 1]$  at random;  
   $p := p \times u;$   
while ( $p > L$ );  
return  $N := k - 1.$ 
```

Donald E. Knuth: *The Art of Computer Programming*, volume 2
Seminumerical Algorithms, 3rd edition, 1998; page 137.

the Poisson distribution in Julia

```
julia> using StatsKit
```

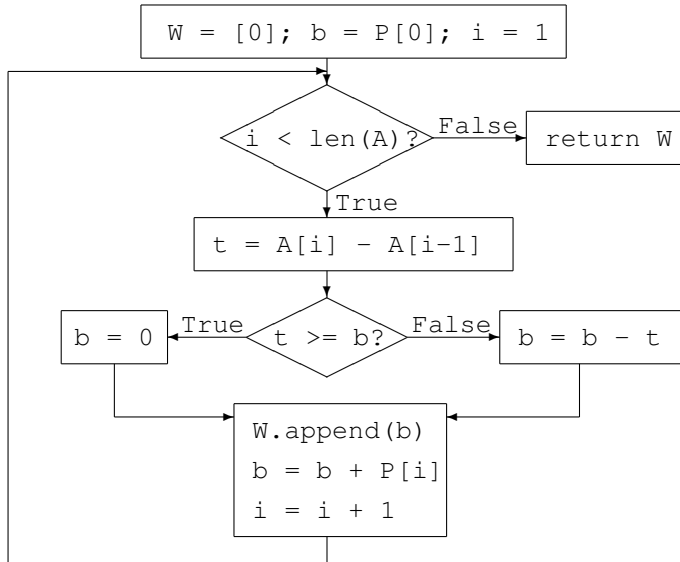
```
julia> p = Poisson(10)
Poisson{Float64} (λ=10.0)
```

```
julia> rand(p, 10)
10-element Vector{Int64}:
```

```
 8
11
 5
11
 6
11
11
12
 9
11
```

```
julia>
```

flowchart for the simulation



the Monte Carlo Method

1

the Monte Carlo Method

- estimating integrals
- histograms of random number generators

2

Mean Time Between Failures

- life span of a multicomponent product
- running simulations

3

Servicing Requests

- estimating wait times
- running simulations

setup of the simulation

Three input parameters:

- 1 The number of arrivals per time step: λ .
- 2 The maximum number of items for each job: M .
- 3 The speed of the processor: time to process one item.

If there are 4 arrivals in one time step,
then there arrival times are 0.0, 0.25, 0.50, 0.75,
simulating the queuing of arrivals, one after the other.

The setup function generates two vectors of the same size:

- 1 The arrival times of the jobs.
- 2 The sizes of the jobs.

a Julia function to process requests

```
"""
```

```
    process_requests(arrivals::Vector{Float64},  
                    jobs::Vector{Int}, speed::Float64)
```

returns the waiting times for the jobs
with given arrival times and sizes.

```
"""
```

```
function process_requests(arrivals::Vector{Float64},  
                        jobs::Vector{Int}, speed::Float64)  
    wait = [0.0 for i=1:length(arrivals)]  
    busy = jobs[1]*speed  
    for i=2:length(arrivals)  
        elapsed = arrivals[i] - arrivals[i-1]  
        if elapsed >= busy  
            busy = 0  
        else  
            busy = busy - elapsed  
        end  
        wait[i] = busy  
        busy = busy + jobs[i]*speed  
    end  
    return wait  
end
```

one run

For the complete program, see the posted code.

The output of one run is below:

```
$ julia estimatewait.jl
Lambda : 10, max#items : 7, speed : 0.02.
Simulation with 1000 trials :
  the average wait time : 0.13
the standard deviation : 0.21
```

multiple processors

The servicing requests problem also models client/server interactions. Servers are multithreaded, so we consider multiple processors.

Exercise 5:

Describe a modification to the servicing requests problem:

- There are m processors, all operating at the same speed.
- If there is no idle processor, then the job is rejected.

Instead of the waiting time,
the modified simulation counts the number of rejected jobs.